Medallion-Architecture Data Pipeline for the Uber NYC Rides Dataset

Technical Project Documentation

GitHub: github.com/shoman042/Uber_End_to_End_Data_Pipeline

1. Introduction

This document describes a data engineering project built around the Uber pickups dataset for New York City, covering the year 2014. The project implements a medallion-architecture pipeline, in which raw trip data is ingested through both batch and streaming paths, progressively cleaned and enriched, and organized into a dimensional data warehouse for analytical querying and reporting.

The project is developed collaboratively, with different team members responsible for different layers or components of the pipeline. The author is responsible for assembling the individual contributions into a single, unified project on a CentOS virtual machine, in addition to implementing specific components of the pipeline directly.

2. Project Overview

The source data consists of eleven monthly Uber raw trip CSV files covering January through November 2014, together with an accompanying manifest file. These files were transferred onto a CentOS virtual machine, on which the necessary data engineering tools are installed, using a VirtualBox shared or dropped folder.

The pipeline follows a medallion architecture composed of Bronze, Silver, and Gold layers, with data flowing through the following two ingestion paths:

- Batch path: historical monthly files are ingested and loaded into the HDFS Bronze layer, then processed with PySpark batch ETL.
- Stream path: the final month of data (December 2014) is used as the source for a simulated live stream, published through Kafka, consumed, and processed with Spark Structured Streaming.

Both paths converge into a unified Silver layer, which in turn feeds a dimensional Gold layer and a Serving layer exposed through HiveQL for downstream consumption, including dashboards, machine learning models, and reports.

3. Scope

The author's assigned responsibility within the project is the streaming path, from ingestion through to the Silver layer. In addition, the author is responsible for assembling the complete project: team members each work on a different layer or component of the pipeline and submit their work to the author, who integrates all contributions into a single project folder on the shared CentOS virtual machine.

Beyond the initial streaming scope, the author's work has extended to include the batch processing path, with the objective of producing a batch-layer output that matches the structure of the stream output so that the two can later be merged into one unified Silver layer.

4. Data Source

The dataset used throughout the project is the Uber pickups dataset for New York City, provided as CSV files for the year 2014. Each trip record contains the following fields: trip identifier, start latitude, start longitude, end latitude, end longitude, start time, end time, and trip distance.

The complete source data consists of eleven monthly raw CSV files (uber_raw_data_01_14.csv through uber_raw_data_11_14.csv) together with a manifest.csv file. The final monthly file, uber_raw_data_12_14.csv, was selected as the source for the simulated Kafka stream.

5. System Architecture

The overall pipeline architecture is illustrated below. Raw data from the Uber pickups dataset enters the system through two parallel paths, a batch path and a stream path, both of which ultimately converge into the Silver, Gold, and Serving layers.

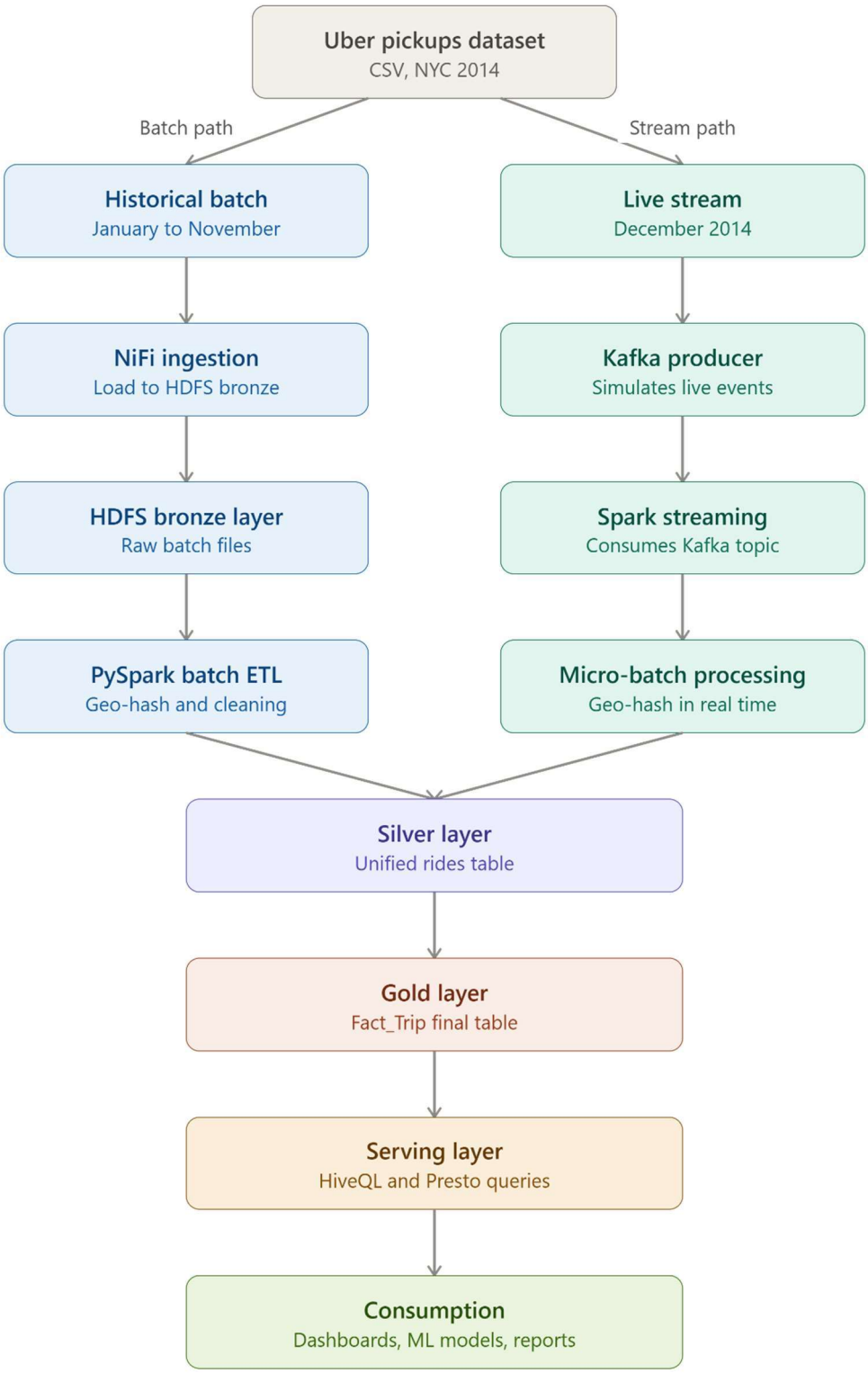

Figure 1. End-to-end pipeline architecture, from raw data ingestion to consumption.

5.1 Batch Path

- Historical monthly data (January to November 2014), per the architecture diagram) is ingested using Apache NiFi.
- Ingested files are loaded into the HDFS Bronze layer as raw batch files.
- A PySpark batch ETL job performs geo-hashing and data cleaning on the Bronze batch data.

5.2 Stream Path

- The live-stream portion of the dataset (December 2014, per the architecture diagram) is published by a Kafka producer that simulates live events.
- A Spark Structured Streaming job consumes the Kafka topic and performs micro-batch processing, including real-time geo-hashing.

5.3 Silver, Gold, and Serving Layers

- The outputs of the batch and stream paths are unified into a single Silver layer containing a unified rides table.
- The Silver layer feeds a Gold layer, structured as a dimensional model culminating in a Fact_Trip table.
- The Gold layer is exposed through a Serving layer, queried using HiveQL .
- Serving-layer data supports downstream consumption in the form of dashboards, machine learning models, and reports.

5.4 Storage Location

All pipeline layers, namely Bronze, Silver, Gold, and the associated checkpoints, are stored on HDFS rather than on local disk. This decision keeps local storage on the virtual machine limited to code, configuration, and documentation, and is consistent with the original architecture design, in which the Bronze layer resides on HDFS.

6. Streaming Component

6.1 Kafka Producer

The Kafka producer reads the December 2014 raw data file and publishes it to a Kafka topic without modifying its content. The pacing and scheduling of message delivery are controlled directly by the producer rather than derived from the original trip timestamps in the source file.

The producer is designed to loop over the source file indefinitely until it is manually stopped, which means the same set of trip identifiers is re-sent on each pass through the file.

6.2 Kafka Consumer

The consumer reads the streamed records from Kafka and writes them as one or more files into the streaming subfolder of the Bronze layer on HDFS. The consumer is designed so that, as soon as a new file appears in this location, the downstream Spark Structured Streaming job detects it and begins processing immediately.

6.3 Spark Structured Streaming Processing

A Spark Structured Streaming job monitors the Bronze streaming folder and processes incoming files into the Silver layer. Because the Kafka producer loops the source file indefinitely, the same trip identifiers are received repeatedly, which would otherwise introduce duplicate records into the Silver layer.

To guarantee zero duplicates in the Silver layer despite an indefinite, looping data source, a time-bounded watermark was judged insufficient, since the producer's loop cycle (approximately eight hours) would exceed any practical watermark window. Instead, deduplication is implemented using a persistent file of previously seen trip identifiers, maintained on HDFS and checked and updated within each micro-batch using Spark's foreachBatch mechanism. This approach deduplicates records across batches indefinitely, independent of elapsed time.

7. Batch Component

The batch component ingests the historical monthly raw CSV files into the Bronze layer on HDFS. At the time of writing, the raw batch data has already been uploaded to the Bronze layer, and the corresponding Spark batch ETL job is in progress.

The batch ETL job is designed to process Bronze batch data into an output shape that matches the structure of the stream path's Silver output. This alignment allows the batch-origin and stream-origin data to later be merged into a single, unified Silver layer, with the two origins kept in separate files within Silver.

8. Batch Ingestion Flow

Batch ingestion into the Bronze layer is implemented as an Apache NiFi flow. The flow lists available source files, fetches their content, checks for duplicate files, and writes the result to HDFS, with dedicated handling for retries and failures.

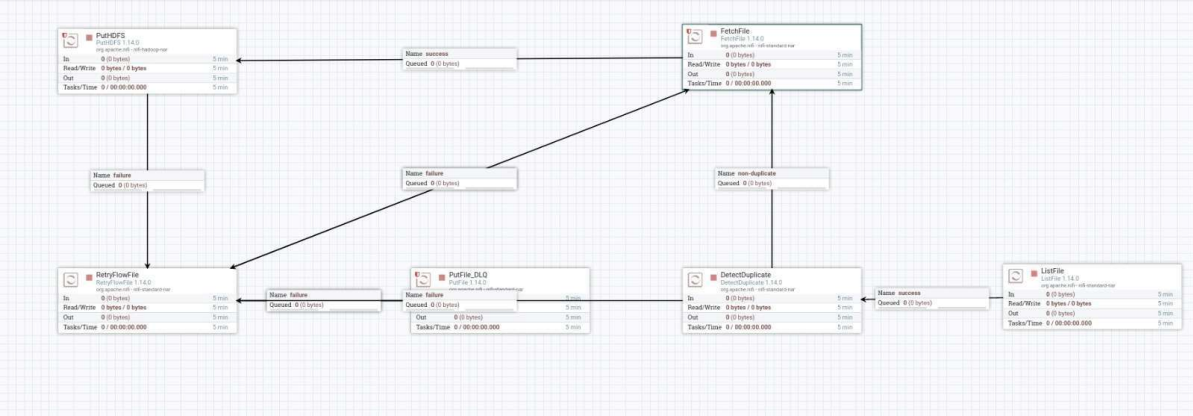

Figure 2. Apache NiFi flow used for batch data ingestion into HDFS.

The flow is composed of the following processors:

- **ListFile:** detects and lists source files to be ingested.
- **DetectDuplicate:** checks incoming files against previously processed files to prevent duplicate ingestion; files identified as non-duplicate proceed to retrieval.
- **FetchFile:** retrieves the content of each file to be ingested.
- **PutHDFS:** writes the retrieved file content into HDFS.
- **RetryFlowFile:** handles files that failed processing, routing them back for another retrieval attempt.
- **PutFile_DLQ:** routes files that repeatedly fail processing to a dead-letter queue location for later inspection.

9. Data Warehouse Design

The Gold layer is structured as a dimensional model, built incrementally on top of the unified Silver layer via a gold_etl.py script. Each dimension uses the same seen-identifier tracking file pattern applied in the streaming deduplication logic, allowing the Gold layer to be built incrementally without introducing duplicate records.

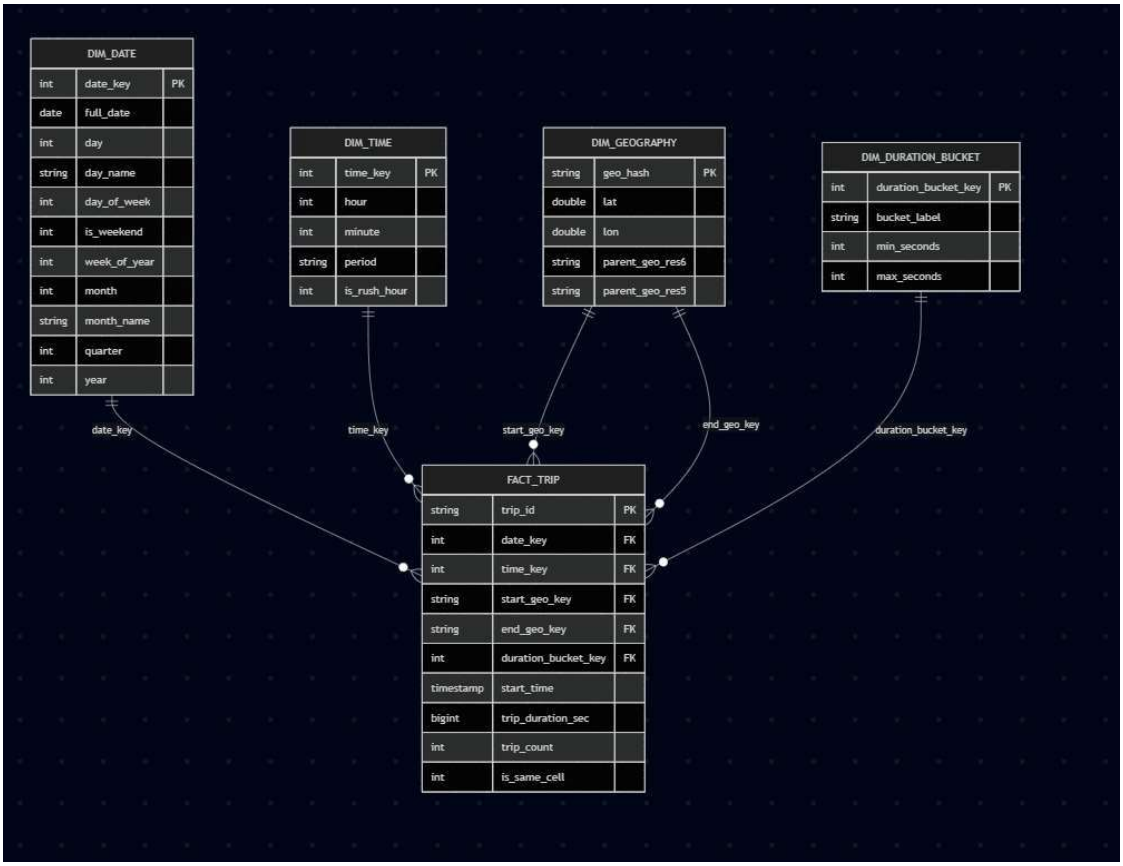

Figure 3. Star-schema data warehouse design for the Gold layer.

9.1 Fact Table

FACT_TRIP is the central fact table, containing the following fields:

- trip_id (primary key)
- date_key (foreign key to DIM_DATE)
- time_key (foreign key to DIM_TIME)
- start_geo_key (foreign key to DIM_GEOGRAPHY)
- end_geo_key (foreign key to DIM_GEOGRAPHY)
- duration_bucket_key (foreign key to DIM_DURATION_BUCKET)
- start_time (timestamp)
- trip_duration_sec (bigint)
- trip_count (int)
- is_same_cell (int)

9.2 Dimension Tables

- **DIM_DATE:** date_key (primary key), full_date, day, day_name, day_of_week, is_weekend, week_of_year, month, month_name, quarter, and year.
- **DIM_TIME:** time_key (primary key), hour, minute, period, and is_rush_hour.
- **DIM_GEOGRAPHY:** geo_hash (primary key), lat, lon, parent_geo_res6, and parent_geo_res5.
- **DIM_DURATION_BUCKET:** duration_bucket_key (primary key), bucket_label, min_seconds, and max_seconds.

The geography dimension is based on a hierarchical geo-hashing scheme, with parent_geo_res6 and parent_geo_res5 representing coarser resolution levels above the base geo_hash. The duration dimension groups trips into labeled duration buckets, each defined by a minimum and maximum number of seconds.

10. Pipeline Orchestration

The pipeline is operated through a control panel that exposes each stage of the pipeline as an individually triggerable action, as well as a single combined action that runs the full pipeline end to end.

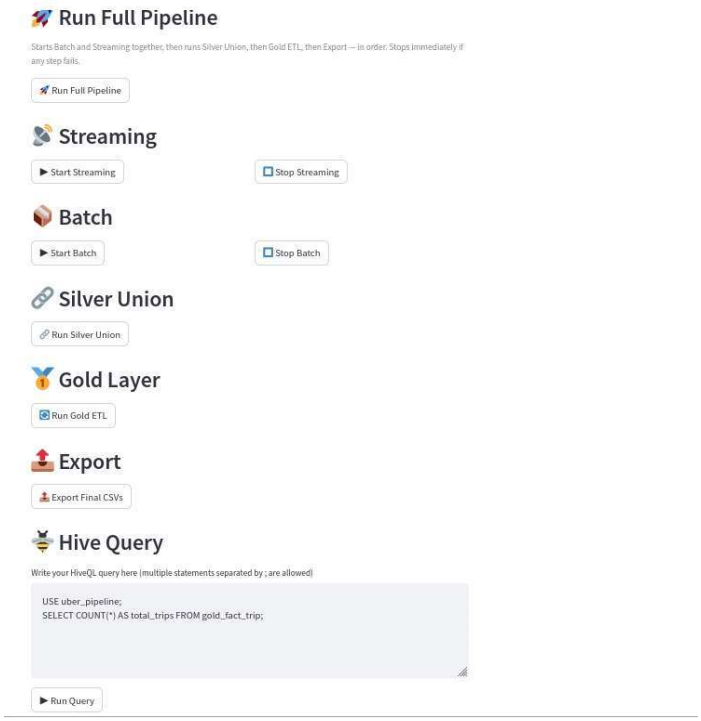

Figure 4. Pipeline control panel, showing individually triggerable stages and a Hive query interface.

The control panel provides the following controls:

- **Run Full Pipeline:** starts the batch and streaming paths together, then runs the Silver union, the Gold ETL, and the export step, in order, stopping immediately if any step fails.
- **Streaming:** start and stop controls for the streaming path.
- **Batch:** start and stop controls for the batch path.
- **Silver Union:** runs the union of the batch-origin and stream-origin Silver data.
- **Gold Layer:** runs the Gold ETL process.
- **Export:** exports the final Gold-layer CSV files.
- **Hive Query:** provides a text interface for writing and running HiveQL queries directly against the pipeline's tables, supporting multiple semicolon-separated statements.

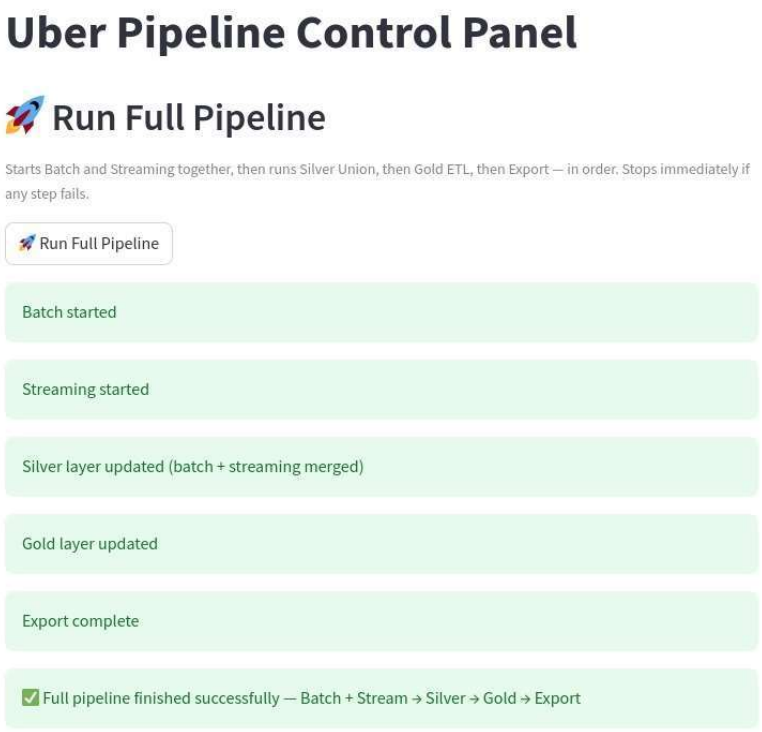

Figure 6. Pipeline control panel, showing individually triggerable stages and a Hive query interface.

11. Serving Layer and Consumption

The Gold layer is exposed for querying through Hive external tables over the Silver and Gold layers, consistent with the Serving layer defined in the overall architecture, and is queried using HiveQL .

The final Gold tables are, in addition, planned for export to local disk as coalesced single files, so that they can be shared with team members for dashboard development.

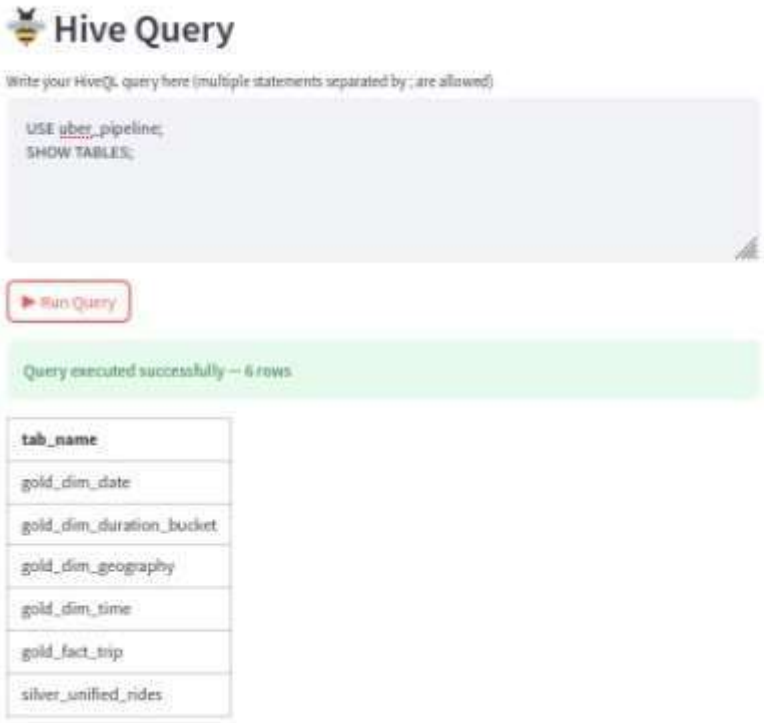

Figure 7. Hive query interface, showing the execution of SQL queries and retrieval of available tables from the data warehouse.

11.1 Analytical Dashboards

The Gold layer data is consumed through a set of analytical dashboards organized into three pages: Overview, Time and Geography, and Duration Analysis.

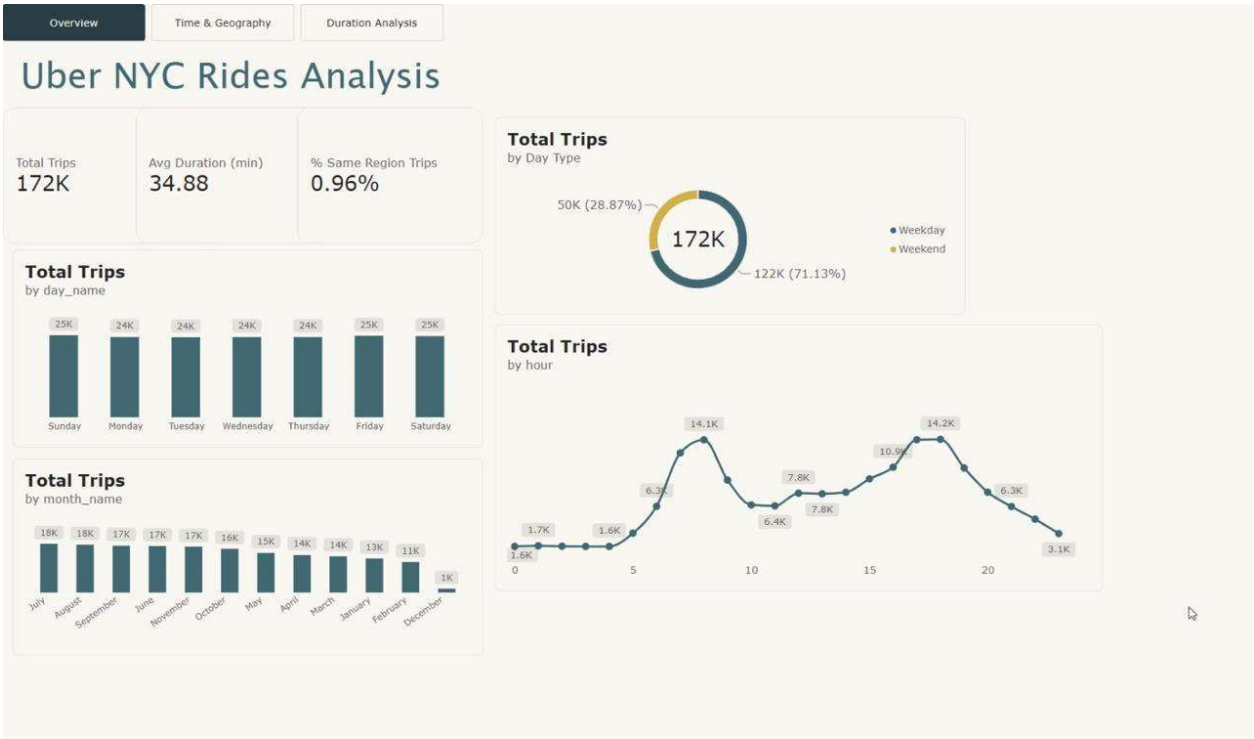

Figure 8. Overview dashboard page, showing total trips, average trip duration, trip distribution by day and month, trip type breakdown, and trips by hour.

Overview Dashboard

The Overview figure provides a high-level summary of the Uber NYC rides dataset through key KPIs and time-based visualizations. It presents the total number of trips (172K), average trip duration (34.88 minutes), and the percentage of same-region trips (0.96%). The dashboard also analyzes ride demand by day type (weekday vs. weekend), day of the week, month, and hour of the day, highlighting the main temporal patterns in trip activity. The hourly trend shows clear demand peaks during the morning and evening, while the day and month charts provide an overview of weekly and seasonal ride behavior.

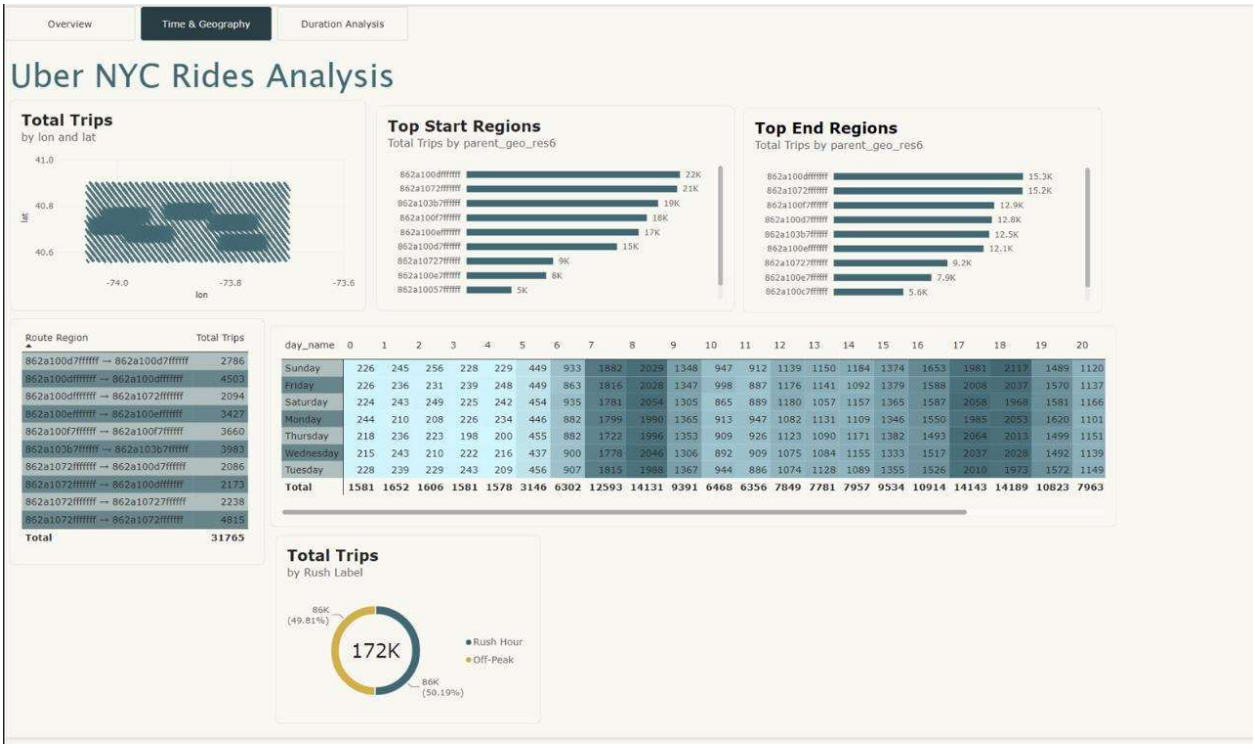

Figure 9. Time and Geography dashboard page, showing a geographic distribution of trips, top start and end regions, a route-region breakdown, and a rush-hour split.

Time & Geography Dashboard:

The Time & Geography page provides a detailed view of Uber trip demand across geographic regions and hours of the day. It shows the overall spatial distribution of trips across NYC, identifies the top pickup and drop-off regions, and highlights the most common routes between geographic areas. The day-of-week × hour matrix reveals how demand changes throughout the week and across different hours, clearly showing stronger activity during weekday commuting periods. The dashboard also compares rush-hour vs. off-peak demand, which is almost evenly split at approximately 49.81% rush-hour and 50.19% off-peak trips, indicating that trips are distributed relatively evenly rather than being concentrated only during peak commuting hours.

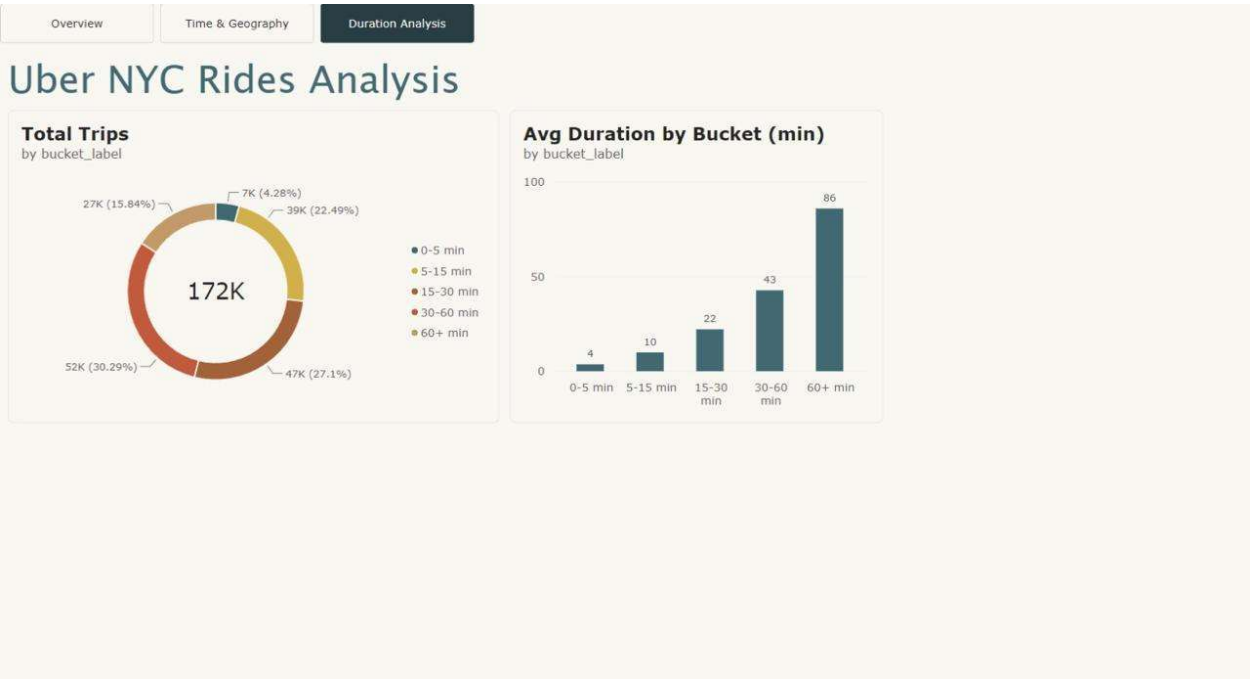

Figure10. Duration Analysis dashboard page, showing the distribution of trips by duration bucket and the average duration per bucket.

Duration Analysis Page

This dashboard page displays a comprehensive analysis of Uber NYC ride durations, segmented into labeled duration brackets.

Total Trips by bucket_label (donut chart): Visualizes the distribution of all 172K trips across specific time categories (0–5 min, 5–15 min, 15–30 min, 30–60 min, 60+ min). A legend ensures correct chronological sorting of the categories. This chart reveals that the majority of rides fall into the 30–60 minute range (30.3%), followed by 15–30 minutes (27.1%). Only a small fraction of rides (4.3%) are very short (under 5 minutes).

Avg Duration by Bucket (min) (bar chart): Shows the average actual trip duration for each bucket. This chart confirms that the bucket boundaries are meaningful, as the average trip length within each category progressively increases, culminating in an average of ~86 minutes for the "60+ min" bracket. This validates the sorting order and the labeling of the duration groups.

12. Project Organization

The project is organized as a single project folder, containing all layers, code, configuration, and documentation for the pipeline. Within the Bronze layer, two dedicated subfolders separate data fed by the stream path from data fed by the batch path.

The project is assembled and maintained on a CentOS virtual machine on which all required data engineering tools are installed. Team members each work on a different layer or component of the pipeline and submit their completed work, which is then integrated into the single shared project folder.

13. Current Status

- The streaming path, including the Kafka producer, Kafka consumer, Spark Structured Streaming job, and deduplication logic, is complete.
- Raw batch data has been uploaded to the Bronze layer on HDFS.
- The batch ETL job, intended to align the batch output with the stream output for a future Silver-layer merge, is in progress.

14. Challenges and Solutions

- **Challenge:** the Kafka producer loops the source file indefinitely until manually stopped, causing the same trip identifiers to be re-sent repeatedly.
- **Solution:** rather than relying on a time-bounded watermark, which would not survive the producer's approximately eight-hour loop cycle, deduplication was implemented using a persistent file of previously seen trip identifiers on HDFS, checked and updated per micro-batch through Spark's foreachBatch mechanism, ensuring zero duplicates in the Silver layer regardless of elapsed time.
- **Challenge:** aligning independently developed batch and stream processing paths so that their outputs can be merged into a single, unified Silver layer.
- **Solution:** the batch ETL job is being built to produce output matching the structure of the existing stream output, and the same seen-identifier tracking pattern used for streaming deduplication is reused for incremental, duplicate-free construction of the Gold-layer dimensions.

15. Conclusion

This project implements a medallion-architecture data pipeline for the Uber NYC 2014 rides dataset, combining batch and streaming ingestion paths into a unified Silver layer, a dimensional Gold layer, and a Hive- based Serving layer. The streaming path, including deduplication of an indefinitely looping data source, has been completed, and work is ongoing to bring the batch path to the same output structure so that the two can be merged. The resulting Gold-layer tables support HiveQL and and feed a set of analytical dashboards covering trip volume, temporal and geographic patterns, and trip duration.